FINAL REPORT

PROJECT NAME:

Electronic Stethoscope

PROJECT TEAM

Oğuzhan Alasulu
Serkan Kıyaklı
Aliye Birgül

Table of Contents

1. **INTRODUCTION** **4**

2. **HEART CIRCUIT** **5**

2.1 ACTIVE LOW-PASS FILTER *5*
2.2 ACTIVE HIGH-PASS FILTER *7*
2.3 CIRCUIT *8*
2.4 WAVEFORM ANALYSIS *11*
2.4.1 MATLAB GUI Signal Acqusitions *11*

3. **RESPIRATORY CIRCUIT** **14**

3.1 WAVEFORM ANALYSIS *14*
3.1.1 MATLAB GUI Signal Acqusitions *14*

4. **HARDWARE & SOFTWARE DESIGN** **16**

4.1 MATLAB GRAPHICAL USER INTERFACE DESIGN *16*
4.2 ALTIUM PCB CIRCUIT DESIGN *23*
4.3 FUSION 360 BOX DESIGN *25*

5. CONCLUSIONS **27**

6. **RESOURCES** **29**

1. Introduction

Sustaining the health of the heart and respiratory system is vital for overall well-being and endurance. Together, they support cellular functions and metabolic processes. Thus, the electronic stethoscope is more ideal than the traditional stethoscope due to providing better sound quality and allowing for more precise diagnosis. The electronic stethoscope is a device that can be used to monitor heart and respiratory signals owing to the different specific circuit types such as filter circuits. For instance, the low-pass Butterworth filter with a cutoff frequency of 734 Hz is used to attenuate unwanted noise from the signals attained from heart and respiratory circuits. This filter is important due to filtering out the signal quality and more accurate diagnosis.

MATLAB Graphical User Interface (GUI) design is another essential apparatus for signal acquisition as it provides a user-friendly interface for acquiring, displaying, and processing signals. The GUI design permits better filtering for signals, which is important for accurate analysis. Additionally, the user can straightforwardly adjust the filter parameters to obtain the desired signal quality through the MATLAB GUI. Moreover, the GUI design allows for real-time monitoring of the signals, which is vital for detecting any abnormalities in the heart and respiratory signals.

Besides, the product design is also essential. The Fusion 360 has been used to design three-dimensional (3D) box design. The 3D box allows for enhanced visualization of the circuit components, which is critical for troubleshooting and maintenance. Furthermore, the PCB design from Altium Designer allows better integration of the electrical circuit components, which is important for diminishing the size of the device.

2. Heart Circuit

2.1 Active Low-Pass Filter

An active low-pass filter is a circuit configuration that allows signals with frequencies below a certain cutoff frequency to pass through while attenuating signals with frequencies above the cutoff. It employs an operational amplifier (op-amp) in its design to actively amplify and filter the input signals. The TL072 is used as operational amplifier for active low-pass filter part.

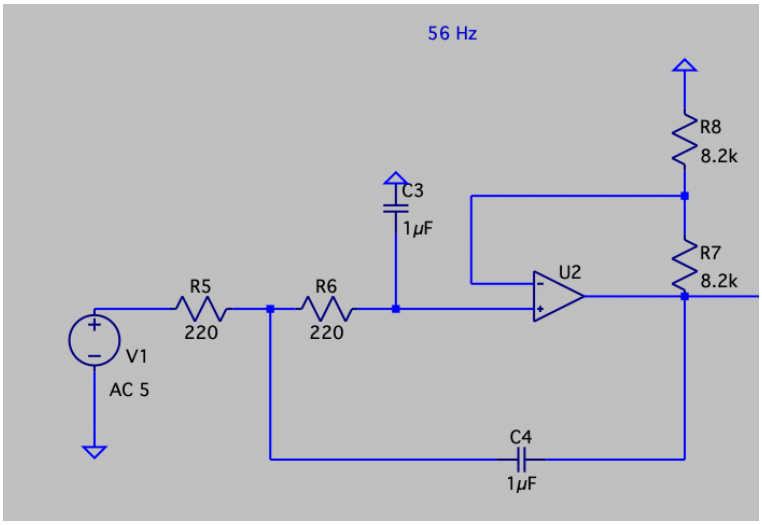

Figure-1 (Low-Pass Filter in LTSpice)

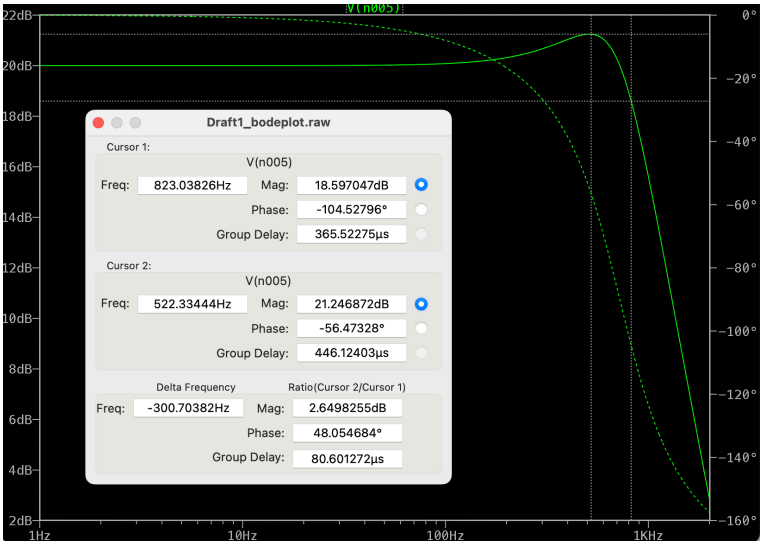

Figure-2 (Low-Pass Filter as Bode Plot in LTSpice)

The Butterworth filter ensures a maximally flat frequency response within its passband, providing a uniform gain up to the cutoff frequency before gradually attenuating the higher frequencies. For the Butterworth filter's component in active low-pass circuit for cut-off

frequency of 56 Hz, the 220Ω resistor and the capacitor of 1µF are used. The values of these components impact on characteristic of active low-pass filter.

$$\boldsymbol{Quality\ Factor\ (Q) = 0.5}$$

$$\boldsymbol{Damping\ Ratio\ (\zeta) = 1}$$

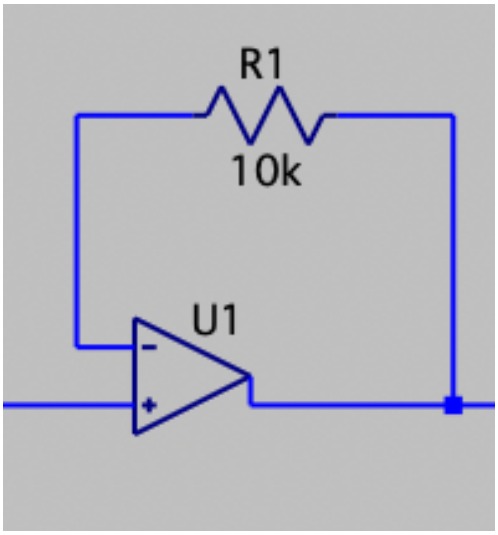

Figure-3 (Buffer Part of Circuit in LTSpice)

Buffer is a device or circuit that is designed to isolate and protect one part of the circuit from the electrical effects of another part. It is commonly used to prevent the loading of a signal source by a connected load, ensuring that the source and load do not significantly affect each other's performance.

Buffer typically consists of an operational amplifier (op-amp) or transistor-based circuit configured to provide high input impedance and low input impedance. The LM358 is used as operational amplifier for buffer part of our circuit. Buffers are used for connection of low-pass filters and high-pass filters of circuit.

2.2 Active High-Pass Filter

An active high-pass filter is an electronic circuit designed to pass signals with frequencies higher than a specified cut-off frequency while attenuating lower-frequency components. This type of filter utilizes the active components, typically operational amplifiers (op-amps), to actively amplify and shape the input signals. The LM358 is used as operational amplifier for active low-pass filter part.

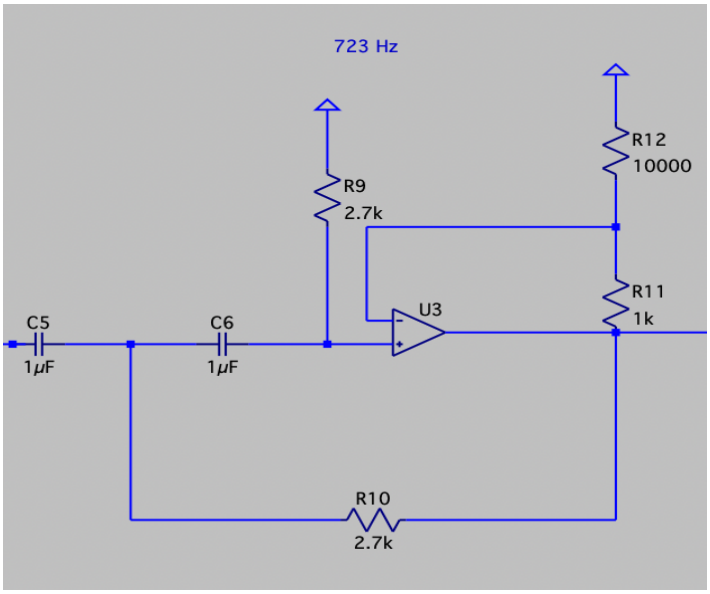

Figure-4 (High-Pass Filter of Circuit in LTSpice)

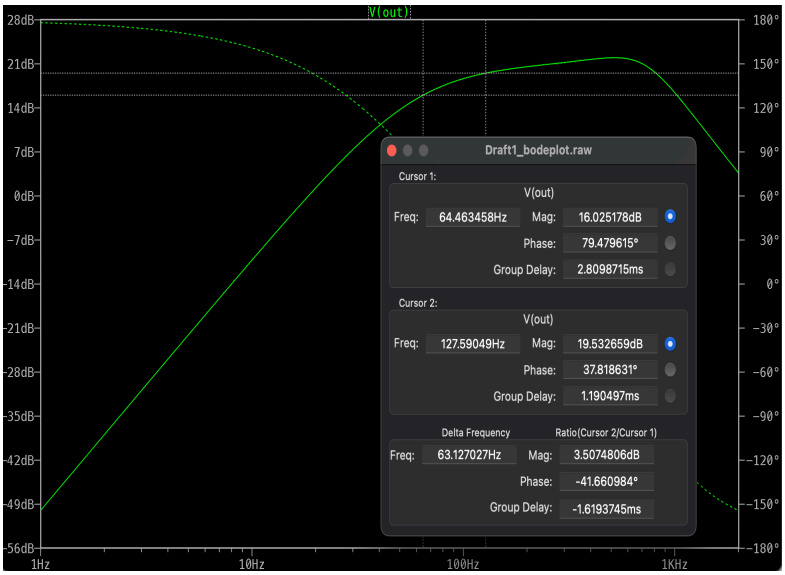

Figure-5 (High-Pass Filter as Bode Plot in LTSpice)

The electric components chosen for the Butterworth filter include a resistor with a value of 2.7 kiloohms and a capacitor with a capacitance of 1 microfarad. These components, in junction with the Butterworth filter topology, determine the time constant of filter, damping factor, quality factor, etc.

$$\textbf{\textit{Quality Factor}}\ (\boldsymbol{Q}) = \mathbf{0.5}$$

$$\textbf{\textit{Damping Ratio}}\ (\boldsymbol{\zeta}) = \mathbf{1}$$

2.3 Circuit

The team participants developed two distinct circuits to address audio signal processing needs. The first circuit, implemented on a breadboard, consist of an LM386 Audio Amplifier along with TL072 op-amps serving as low-pass and high-pass filters. LM358 op-amps are strategically placed between the filters and the amplifier, providing buffer functionality. This configuration allows for signal conditioning, filtering, and subsequent amplification on a single breadboard setup.

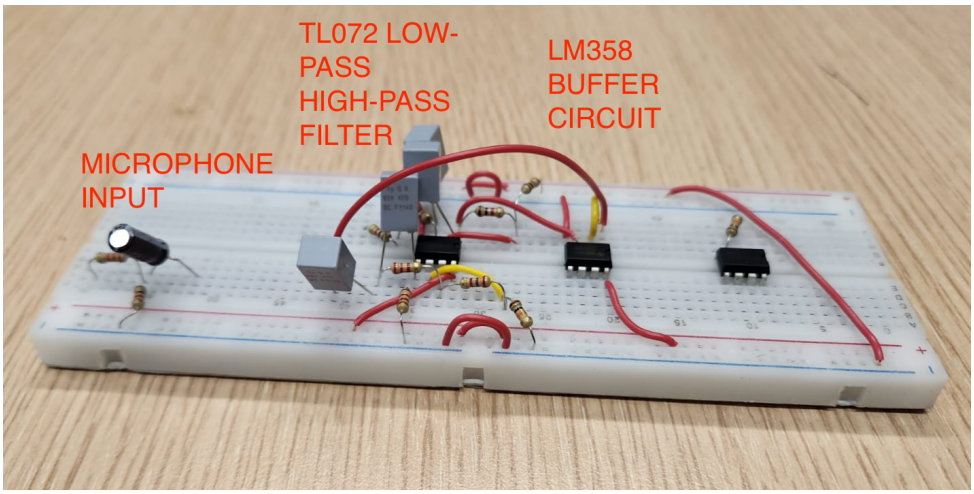

Figure-6 (Original Circuit in Breadboard)

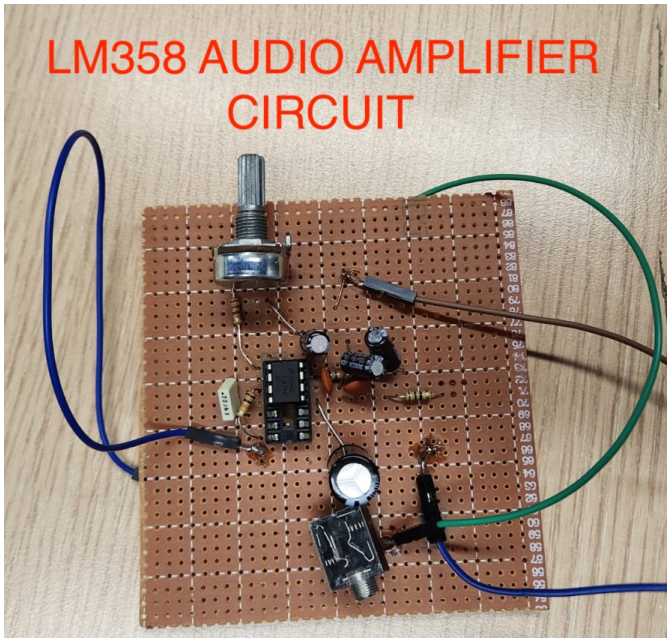

Figure-7 (LM358 Audio Amplifier Circuit)

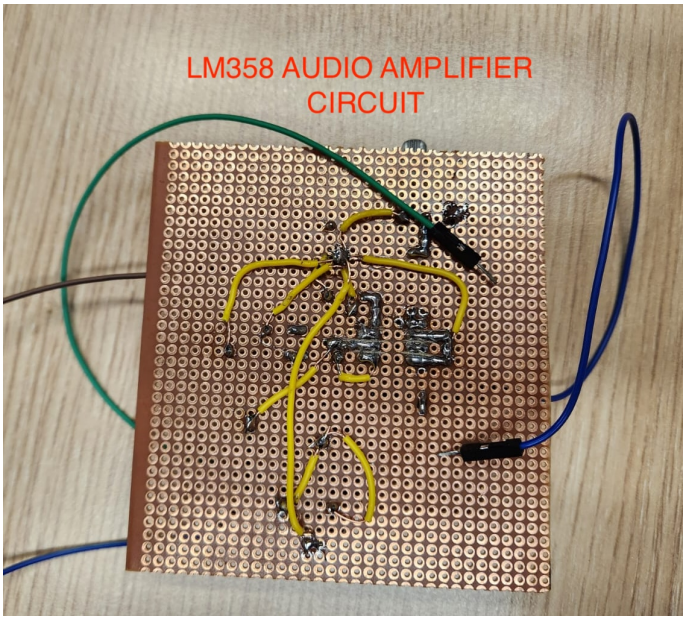

Figure-8 (LM358 Audio Amplifier Circuit Side View)

In contrast, the second circuit is designed as a more compact solution on a printed circuit board (PCB). This PCB circuit features a streamlined layout, incorporating LM358 op-amps for buffering, followed by TL072 op-amps for low-pass and high-pass filtering stages. Similar to breadboard setup, LM386 serves as the audio amplifier at the final stage. The integration of these components on a single circuit card enhances the compactness.

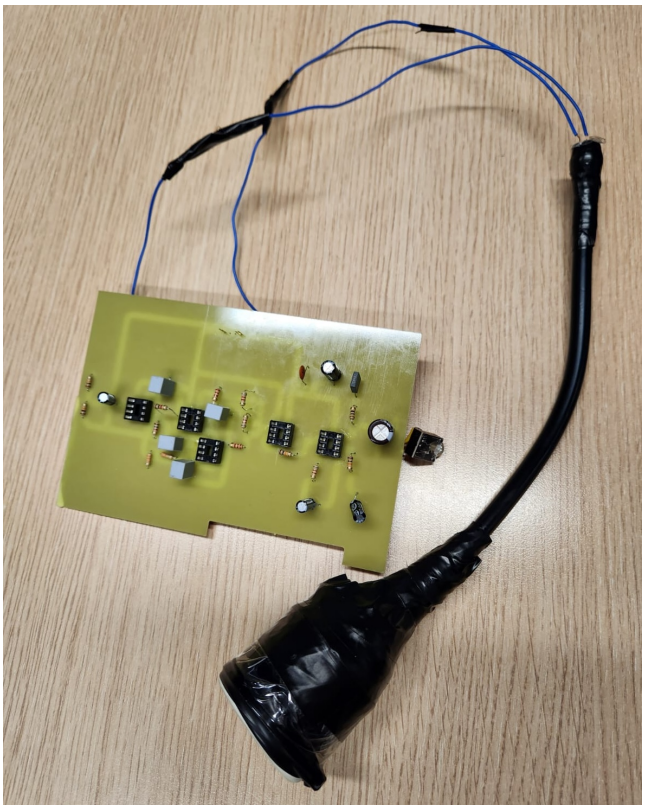

Figure-9 (PCB Circuit with Handmade Transducer)

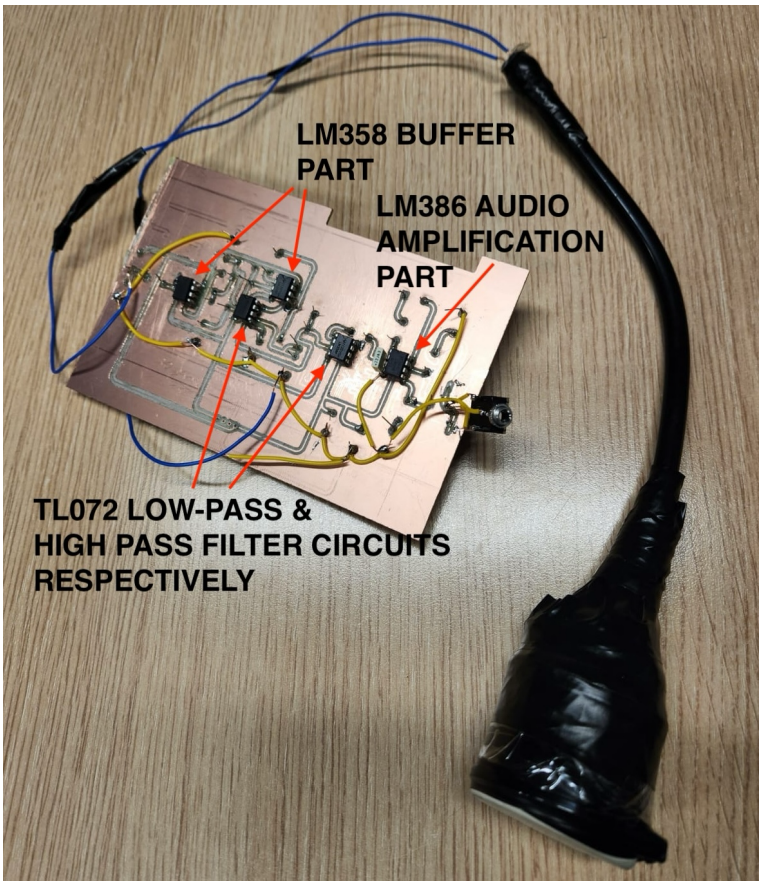

Figure-10 (PCB Circuit with Handmade Transducer)

2.4 Waveform Analysis

Waveform analysis states to the inspection and interpretation of the profile, features, and patterns of a signal or wave over time. In the context of heart signal acquisition, waveform analysis involves reviewing the graphical illustration of the heart's electrical activity over a specific time period. The electrodiogram (ECG or EKG) is a common depiction of the heart's electrical waveform.

The significance of waveform analysis in heart signal acquisition lies in its capability to provide detailed insights into the cardiac cycle and classify various pieces of heart function. Key components of the ECG waveform include the P wave, QRS complex, and T wave, each representing particular segments of the cardiac cycle. By analyzing these components, healthcare experts can assess heart rate, rhythm, transmission delays, and abnormalities, aiding in the diagnosis and monitoring of cardiac situations. In order to get this waveform dynamics, MATLAB® provides comprehensive options and functions through the GUI (graphical user interface).

2.4.1 MATLAB® GUI Signal Acquisitions

MATLAB® GUI Application is designed for acquiring and evaluating heart and respiratory signals. Distinct visual inconsistencies are detected between sick and healthy people in both raw and filtered signal domains. The raw signals display irregularities and anomalies in sick individuals, while the filtered signals enhance the clarity of relevant features. Moreover, the Fast Fourier Transform (FFT) graphs of the signals demonstrate noteworthy differences in the frequency domain for both healthy and unhealthy individuals. Hereby, FFT provides valuable insights into the variations associated with different health conditions. This comprehensive analysis highlights the possible signals as indicators of health status, emphasizing the efficacy of our MATLAB GUI application in distinguishing between sick and healthy issues. In this step, to comprehend the attribute of our heart signal, MATLAB® GUI benefits will be covered and observed. The intention is obtaining the clear heart signal with frequency value lower than 100 Hz. The sampling frequency of the system is 1390 Hz.

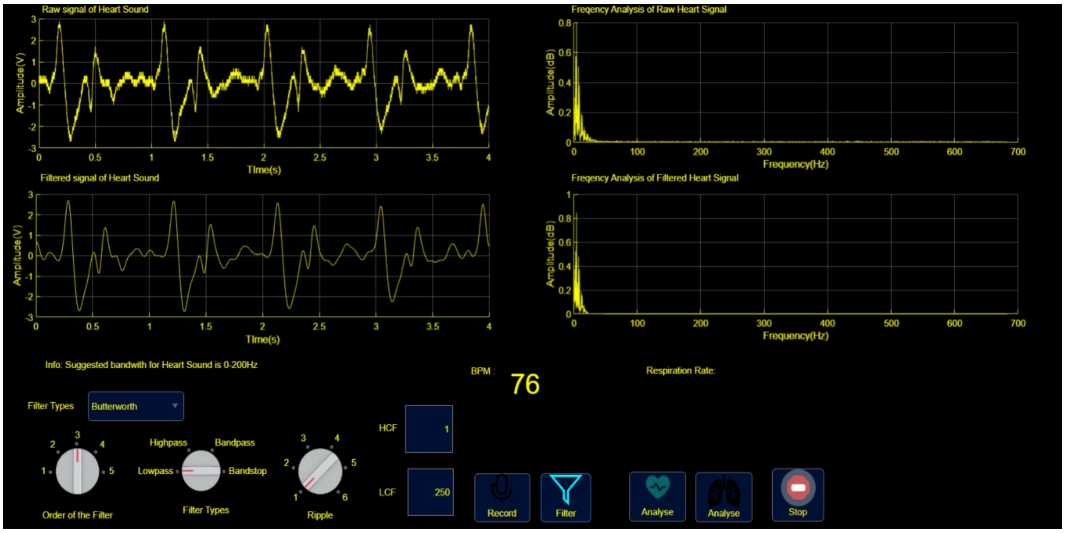

Figure-11 (Aorta Raw & Filtered Heart Signals with FFTs of Healthy Person Observed by Breadboard Circuit)

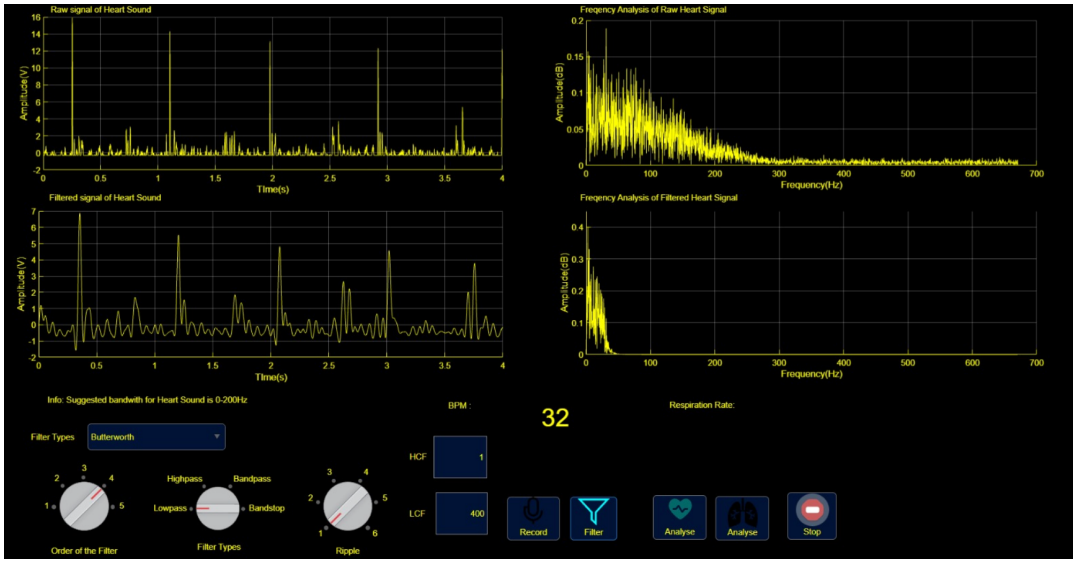

Figure-12 (Raw & Filtered Heart Signals with FFTs of Healthy Person Observed in Chest by PCB Circuit)

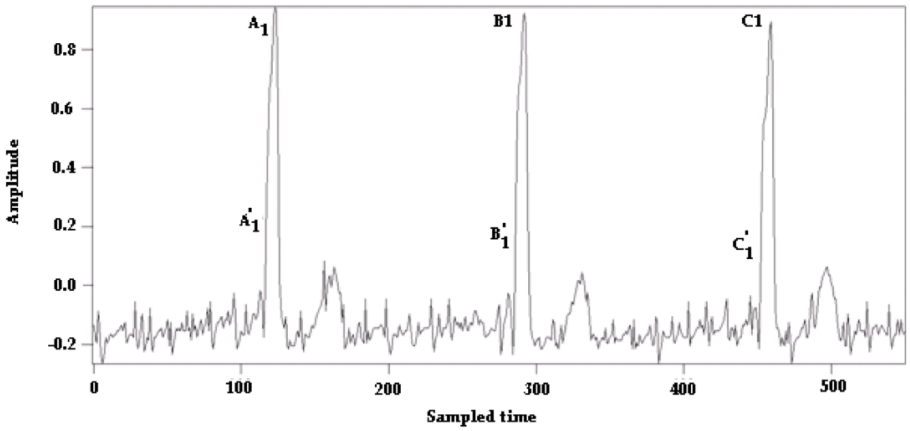

Figure-13 (An Abnormal ECG Signal obtained from a patient with a supraventricular arrhythmia) [Dliou et al.]

Heart signals can be used to discriminate between healthy and diseased individuals. According to a study by Hossain et al, the average frequency of heart sounds is sufficiently discriminatory to differentiate between a healthy and unhealthy heart. The study also established that the variance of a heart sound can be used to differentiate between healthy and unhealthy people. If the variance of heart sound is above 0.015, it is unhealthy, while if the variance value is less than 0.01, it will be marked as healthy. Obviously, Figure-11 and Figure-12 show the noise differences between each other. In that way, for sharper predictions, filtering the signal stays as the critical step.

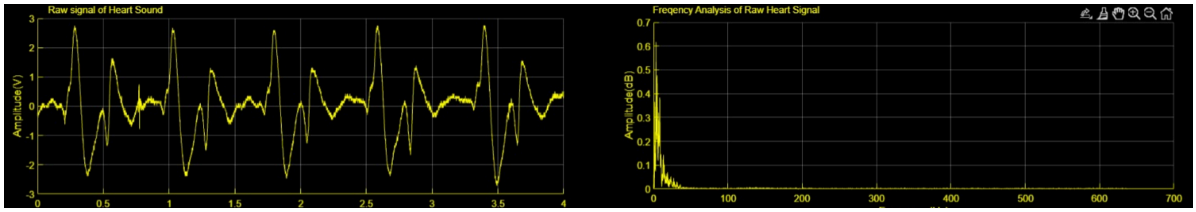

Figure-14 (Raw Aorta Signal & FFT Graph)

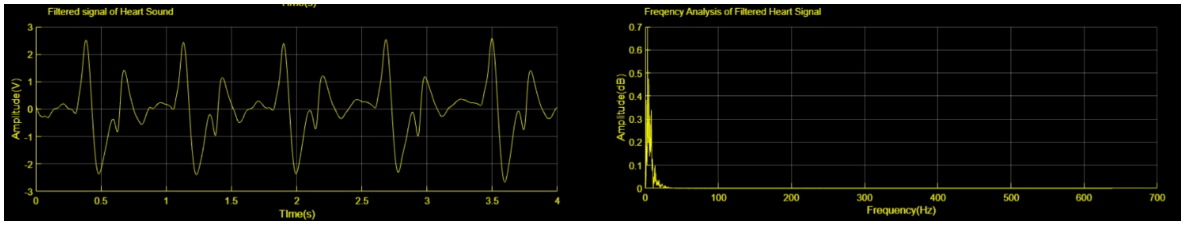

Figure-15 (Filtered Aorta Signal & FFT Graph)

A raw heart signal characteristically covers inherent noise, pieces, and intervention, making it challenging to recognize original patterns. Raw signals may comprise electrical interference, baseline drift, or motion artifacts, which can obscure the true physiological information. On the other hand, a filtered heart signal undergoes signal processing techniques aimed at removing undesirable noise and enhancing relevant features. As shown in the Figure-14, filter cleaned out all the explained drawbacks that contained in Figure-15. Filtering helps reveal the true rhythmic pattern of the heartbeat, making it easier to identify abnormalities or in the cardiac signal.

3. Respiratory Circuit

3.1 Waveform Analysis

Waveform analysis involves the study and interpretation of the graphical representation of a signal or wave over time. In the context of respiratory signal acquisition, waveform analysis pertains to the examination of patterns in signals that represent respiratory activities. The respiratory signal can be obtained through various methods, such as spirometry, plethysmography, or chest wall movement monitoring.

The importance of waveform analysis in respiratory signal acquisition lies in its ability to provide valuable data about a person's breathing patterns and respiratory health. By examining the shape and characteristics of the respiratory waveform, healthcare professionals can gain insights into factors such as respiratory rate, tidal volume, inspiratory and expiratory times, and overall respiratory dynamics. This information is vital for assessing lung function, spotting respiratory syndromes, and monitoring the effectiveness of interventions or treatments. In order to get this waveform dynamics, MATLAB® provides comprehensive options and functions through the GUI (graphical user interface).

3.1.1 MATLAB® GUI Signal Acquisitions

MATLAB® GUI application for this part is designed for respiratory signal acquisition respect to given cut-off frequency that is provided by the user. GUI revealed distinct graphical differences between sick and healthy individuals in both raw and filtered signals. Raw breathing signals from sick participants presented anomalies and differences, contrasting with smoother shapes detected in healthy subjects. The application’s implementation of noise decreasing practices in filtered signals enhanced clearness, facilitating the identification of health-related abnormalities. Furthermore, the Fast Fourier Transform (FFT) analysis uncovered visually distinct frequency features between sick and healthy subjects, proposing valuable comprehensions intro respiratory variations associated with different health conditions. This part highlights the potential respiratory issues, analyzed through our MATLAB GUI application, as effective indicators of health status. The intention is obtaining the clear heart signal with frequency value between 100 Hz and 900 Hz. The sampling frequency of the system is 1390 Hz.

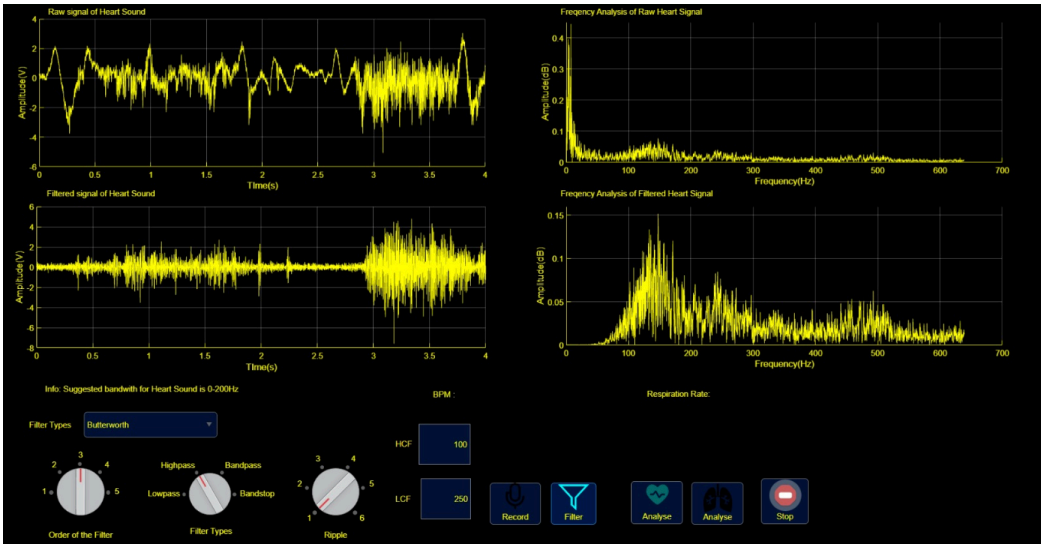

Figure-16 (Aorta Raw & Filtered Respiratory Signals with FFTs of Healthy Person)

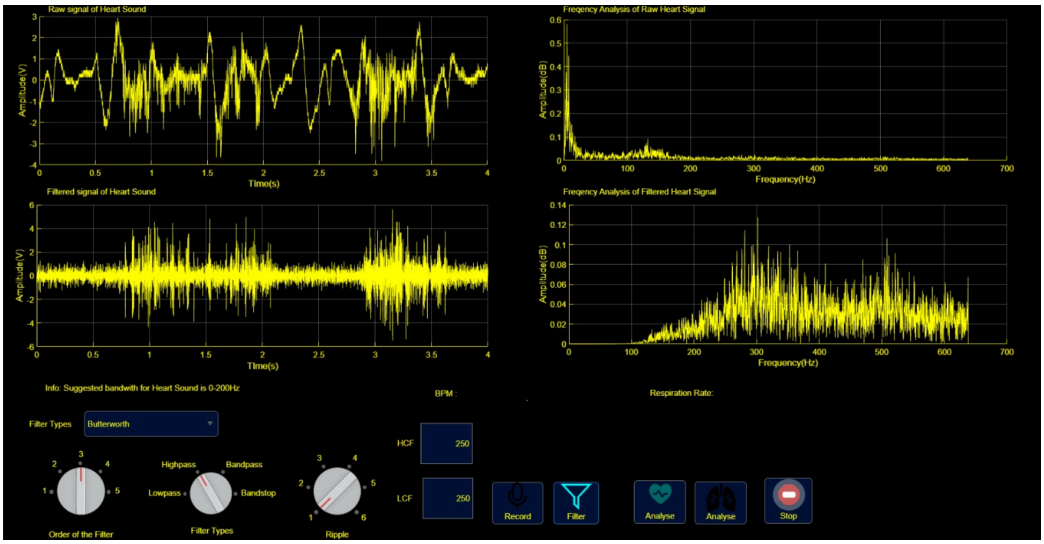

Figure-17 (Aorta Raw & Filtered Respiratory Signals with FFTs of Healthy Person)

This study investigates the analysis of raw and filtered respiratory signals in aortic sounds, pointing to differentiate between healthy and sick individuals. Observations reveal sharp noise and crackling in raw signals from sick individuals, considerably reduced through filtering. Frequency analysis using the Fast Fourier Transform (FFT) depicts a typical peak at 300Hz for sick individuals, compared to 150 Hz in healthy equivalents. Additionally, the sick individuals exhibit a broader frequency spectrum extending up to 900 Hz. These findings propose aortic sounds as a potential indicator of diagnostic markers for respiratory health assessment.

4. Hardware & Software Design

4.1 MATLAB® GRAPHICAL USER INTERFACE DESIGN

In project participants' endeavors, MATLAB appeared as the preferred tool for its robust proficiencies in signal processing and GUI app design. Leveraging MATLAB, project participants efficiently filtered signals, unveiling their FFT characteristics, and augmenting visualization by defining low and high cutoff intervals. The flexibility of MATLAB's App Designer demonstrated instrumental, providing an advantage in constructing user-friendly interfaces. With extensive callback and property options, MATLAB App Designer modernized the development process, ensuring a seamless user experience and simplifying dynamic interactions with the signals under analysis.

Furthermore, to ease the analog reading of voltage values, team participants integrated Arduino into circuit system. Arduino served as a consistent platform for acquiring raw signal values, allowing for seamless transportation of data to MATLAB. This integration not only extended the scope of circuit data acquisition capabilities but also boosted the overall efficiency of signal processing of circuit. The collaboration between MATLAB and Arduino emphasized the comprehensive approach taken by team members, combining the assets of both tools to achieve a more nuanced and insightful evaluation of respiratory signals. For better understanding, some critical parts for GUI are briefly exemplified and explained at below with GitHub link to reach all GUI at the end. At the final, the proof for FFT works well is stated.

```
properties (Access = public)


    s = serialport("COM4", 230400);

    save_path = "C:\Users\Asus\Desktop\Yeni klasör\SAMPLING\RECORDING"


    filtering = 0;
    EEG;
    time;
    recording = true;
    b;
    a;
    EEGfilter;
    heartrate;
    system = 0;
    last_ECG;
    fs;
    bpmvalue;
    record;

end
```

Figure-18 (Properties Part of GUI MATLAB® Code)

In MATLAB®, properties refer to the characteristics or attributes of graphical items, for instance figures, axes, or user interface elements shaped using MATLAB®'s App Designer or GUI-building functions. These properties outline the appearance, behavior, and further aspects of the objects. The properties of GUI code and serial port details are given with baud rate and USB port in Figure-18.

```
try
    data = readline(app.s);
catch exception


end
frame = 4000;
app.EEG = zeros(1, 1000000);
app.time = zeros(1, 1000000);
i = frame;
app.record = zeros(1, 1000000);

plotECG = plot(app.EEGplot, 0, 0, 'y');  % Initialize plot with dummy data
plotECGFFT = plot(app.EEGFFT, 0, 0, 'y');  % Initialize plot with dummy data
plotEEGfiltered = plot(app.EEGplotfiltered, 0, 0, 'y');
plotEEGFFT_2 = plot(app.EEGFFT_2, 0, 0, 'y');
```

Figure-19 (Data Acquiring and Recording Codes)

The provided codes in Figure-19 demonstrate the process of obtaining data from a circuit and recording it.

```
while app.recording
    %tic
    for j = 1:100
        i = i + 1;
        data = str2double(strsplit(readline(app.s), ","));
        app.EEG(i) = data(1);
        app.time(i) = data(2);

    end
    app.fs = frame * 1000000 / (app.time(i) - app.time(i - frame));
    app.last_ECG = app.EEG(i - frame + 1:i);
    ECG_normalized = zscore(app.last_ECG);
    time_vector = linspace(0, 4, frame);
    % Plot the ECG signal
    set(plotECG, 'XData', time_vector, 'YData', ECG_normalized);
    %disp(app.fs)
    % Plot the one-sided FFT
    % N = 2^12;
    L = frame;
    fft_result = abs(fft(normalize(app.last_ECG)));
    %app.record = app.EEG;
    P2 = abs(fft_result/frame);
    P1 = P2(1:L/2+1);
    P1(2:end-1) = 2*P1(2:end-1);
    f = app.fs/L*(0:(L/2));
```

Figure-20 (Data FFT Codes)

The provided codes in Figure-9 illustrate the procedures for processing data acquired from a circuit and generating its Fast Fourier Transform (FFT). These instructions likely include algorithms or functions to analyze the acquired data.

```
differnce = mean(diff(locs_bpm));
seconds = mean((differnce*7)/4000);
bpm = (60/seconds) + 1;
app.bpmvalue = mean(round(mean(round(bpm))));

  app.BPM.Text = num2str(app.bpmvalue);
```

Figure-21 (BPM Calculation Codes)

The provided codes in Figure-21 outline the steps for computing the beats per minute (bpm) value from data obtained from the circuit.

```
app.filtering = 1;
app.FilterImage.ImageSource = 'filteron.svg';
switch app.FilterTypesDropDown.Value
    case 'Butterworth'
        switch app.FilterTypesKnob.Value
            case 'Lowpass'
                [app.b, app.a] = butter(str2double(app.OrderoftheFilterKnob.Value), ...
                    app.LCFEditField.Value/(19200/2), 'low');
            case 'Highpass'
                [app.b, app.a] = butter(str2double(app.OrderoftheFilterKnob.Value), ...
                    app.HCFEditField.Value/(1440/2), 'high');
            case 'Bandpass'
                [app.b, app.a] = butter(str2double(app.OrderoftheFilterKnob.Value), ...
                    [app.HCFEditField.Value/(1440/2) app.LCFEditField.Value/(1440/2)], 'bandpass');
            case 'Bandstop'
                [app.b, app.a] = butter(str2double(app.OrderoftheFilterKnob.Value), ...
                    [app.HCFEditField.Value/(1440/2) app.LCFEditField.Value/(1440/2)], 'stop');
        end
    case 'Chebyshev'
        switch app.FilterTypesKnob.Value
            case 'Lowpass'
                [app.b, app.a] = cheby1(str2double(app.OrderoftheFilterKnob.Value), ...
                    str2double(app.RippleKnob.Value), app.LCFEditField.Value/(1440/2));
            case "Highpass"
                [app.b, app.a] = cheby1(str2double(app.OrderoftheFilterKnob.Value), ...
                    str2double(app.RippleKnob.Value),app.HCFEditField.Value/(1440/2), 'high');
            case 'Bandpass'
                [app.b, app.a] = cheby1(str2double(app.OrderoftheFilterKnob.Value), ...
                    str2double(app.RippleKnob.Value), [app.HCFEditField.Value/(1440/2)...
                    app.LCFEditField.Value/(1440/2)], 'bandpass');
            case 'Bandstop'
                [app.b, app.a] = cheby1(str2double(app.OrderoftheFilterKnob.Value), ...
                    str2double(app.RippleKnob.Value), [app.HCFEditField.Value/(1440/2) ...
                    app.LCFEditField.Value/(1440/2)], 'stop');
        end
end
```

Figure-22 (Filter Codes for Butterworth and Chebyshev)

The provided codes in Figure-22 demonstrate the process of filtering signals acquired from the circuit.

GitHub Link:

https://github.com/alasulu/SAPA_Project.git

To validate the effectiveness of MATLAB® code, we conducted assessments using a frequency tone generator. By inputting the specific frequency values and applying the designed filter, team participants aimed to demonstrate the robust functionality of MATLAB® code in correctly processing signals. This practical confirmation ensures that MATLAB® algorithms perform well under controlled conditions and reinforces the reliability of our approach in real-world scenarios.

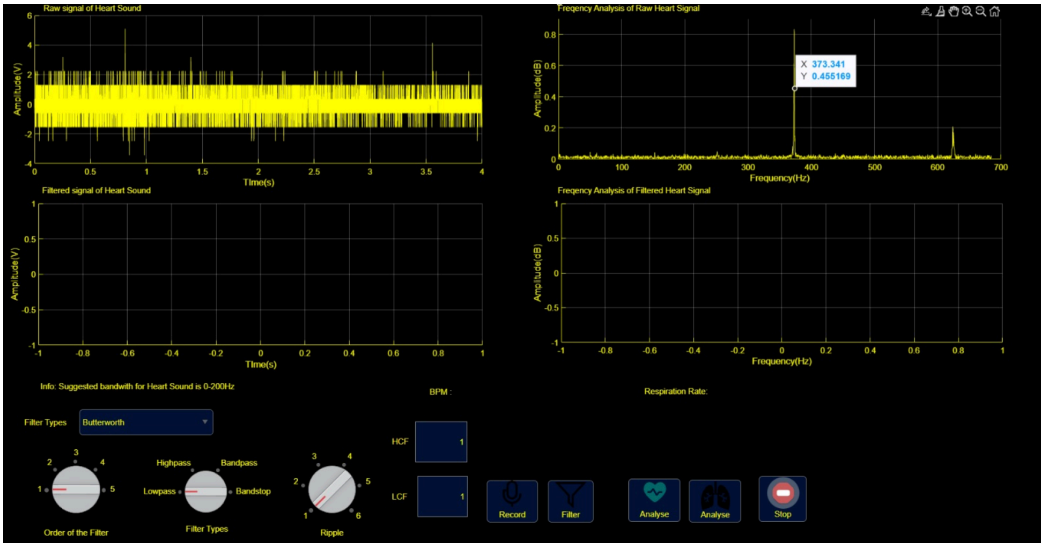

Figure-23 (Low-Pass 370 Hz Tone Plot Graph)

Figure-23 visually represents the impact of a tone generator set at 370 Hz with a low-pass filter. The signal at 370 Hz is prominently displayed, indicating successful signal processing. Notably, beyond 370 Hz, signals appear amplified. Conversely, below the 370 Hz mark, signals exhibit attenuation, demonstrating the effective low-pass filtering characteristics of the applied filter.

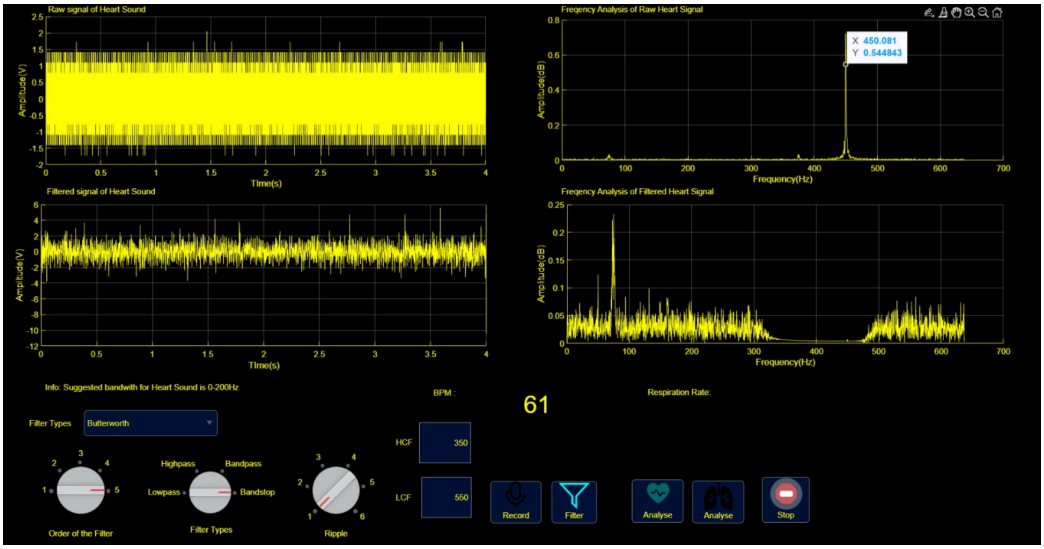

Figure-24 (Band-Stop between 350-550 Hz Tone Plot Graph)

Figure-24 depicts the influence of tone generator at 350 Hz in first FFT plot. In second FFT plot, a band-stop filter is employed within the frequency of 350-550 Hz. In this representation, signals below 350 Hz are clearly visible, as are signals beyond 550 Hz. Notably, within the specified band (350-550 Hz), signals appear attenuated indicating the effective blocking of frequencies within this range.

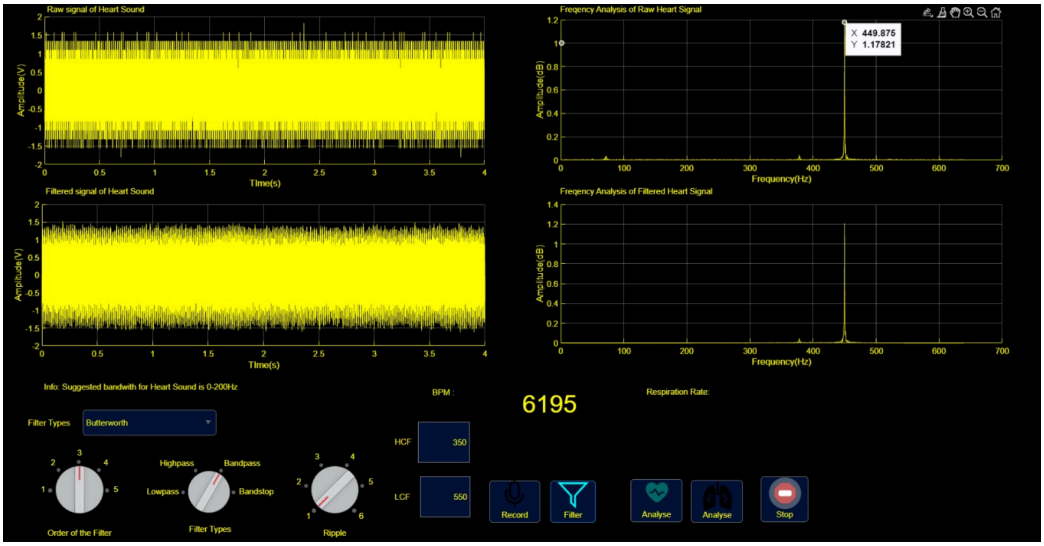

Figure-25 (Band-Pass with Cutoff of 350-550 Hz Tone Plot Graph)

Figure-25 illustrates the effects of tone generator operating within the frequency of range of 350-550 Hz, utilizing a band-pass filter. In this representation, signals below 350 Hz and beyond 550 Hz are not visible, indicating successful attenuation outside the specified frequency band.

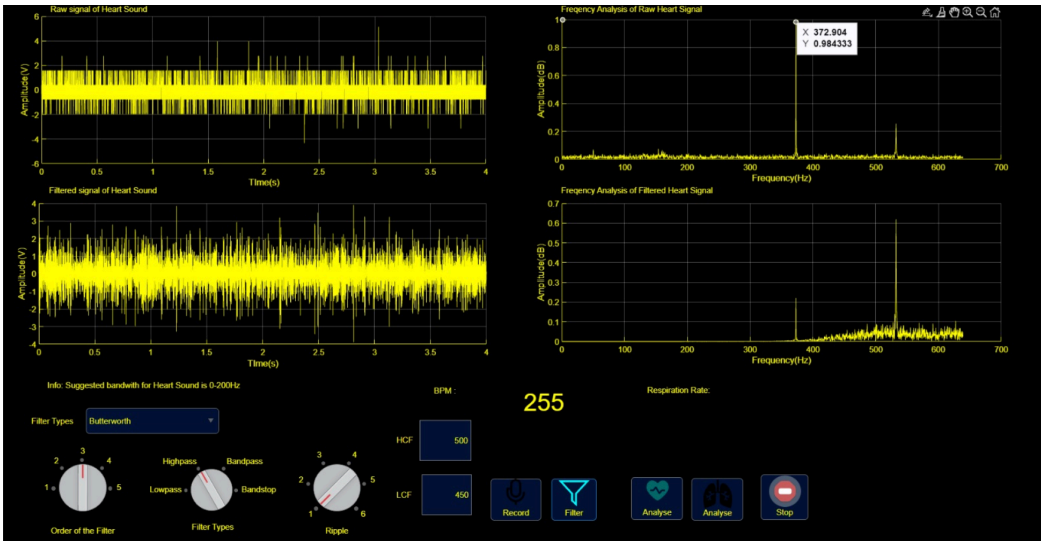

Figure-26 (High-Pass with Cutoff of 370 Hz Tone Plot Graph)

Figure-26 depicts the impact of tone generator set at 370 Hz with a high-pass filter featuring a specified cut-off frequency of 500 Hz. In this representation, signals below 500 Hz are not so visible up to 370 Hz. Below the 370 Hz, it is completely attenuated, demonstrating the successful attenuation of frequencies beneath the designated threshold. Conversely, signals above 500 Hz are amplified.

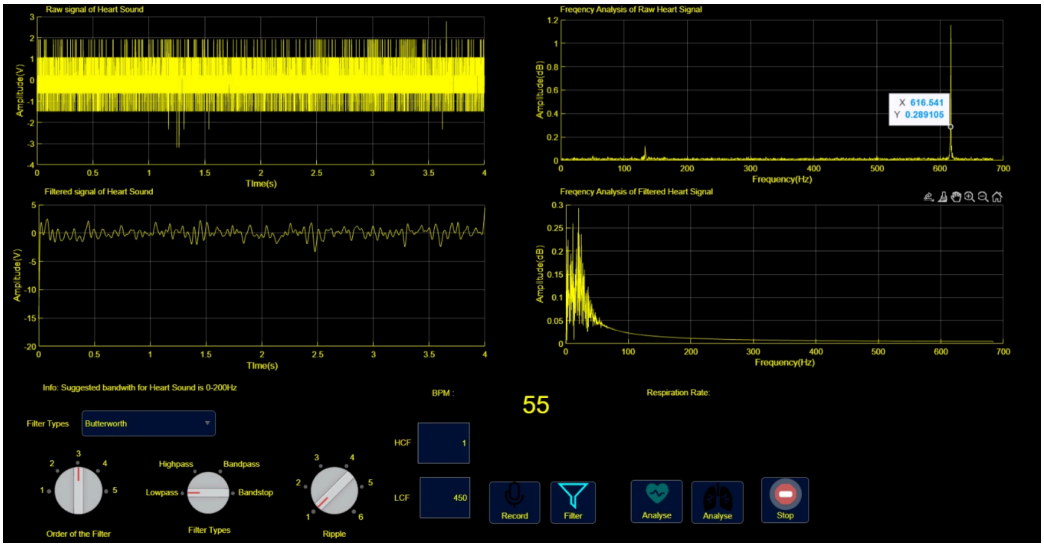

Figure-27 (Low Pass of 610 Hz Tone Plot Graph)

Figure-27 illustrates the effect of Tone Generator operating at 610 Hz with a low-pass filter featuring a specified cut-off frequency of 450 Hz. In this representation, signals below 450 Hz are visible and exhibit amplification. Beyond 450 Hz cut-off, signals are attenuated.

4.2 ALTIUM PCB CIRCUIT DESIGN

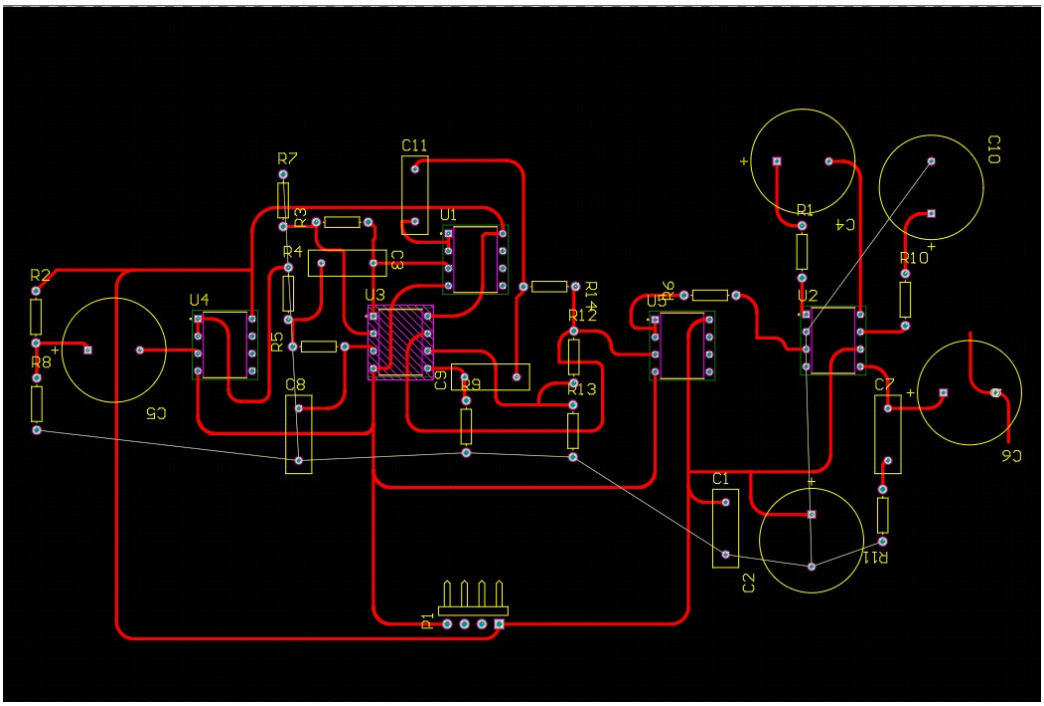

Figure-28 (Low Pass of 610 Hz Tone Plot Graph)

In the Altium PCB design, a comprehensive signal processing circuit has been implemented. The design includes five operational amplifiers (opamps), with LM358 (U4 in sketch) serving as the buffer circuit component. The signal then enters a filtering section that utilizes TL072 (U3) opamps to implement a band-pass filter spanning a frequency range of 56-723 Hz.

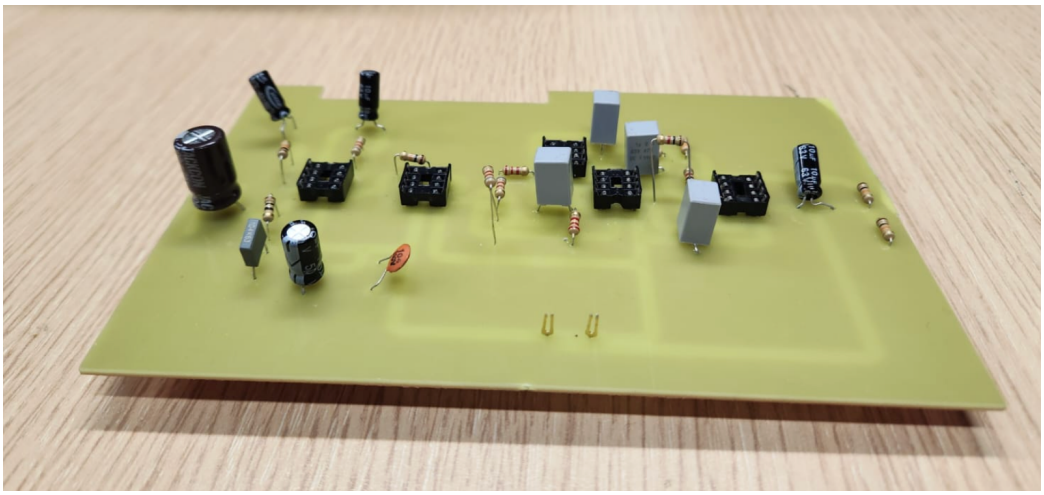

Figure-29 (Low Pass of 610 Hz Tone Plot Graph)

Following the filtering stage, the signal passes through LM358 (U1) opamp acting as buffer component situated between low-pass and high-pass filters, effectively managing the signal flow between the inputs and outputs of the TL072 (U3) opamps.

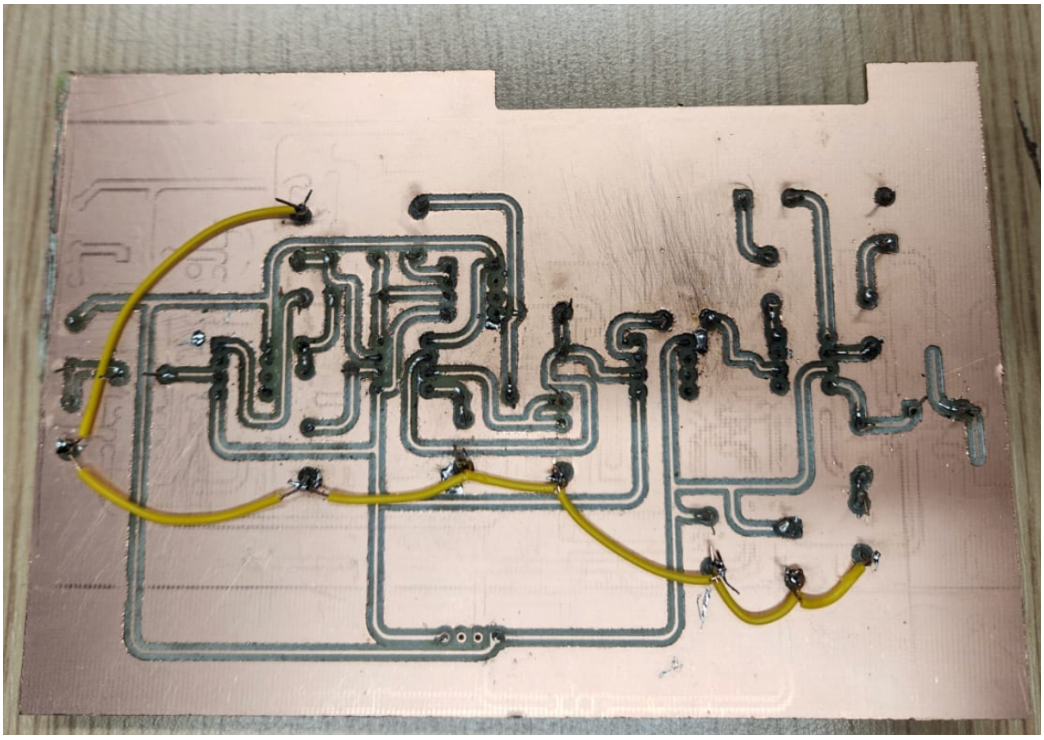

Figure-30 (Low Pass of 610 Hz Tone Plot Graph)

The high-pass output from the TL072 is then processed through another buffer stage implemented with an LM358 (U5) opamp. Finally, the signal reaches an audio amplification stage using the LM386 (U2), providing the necessary amplification for the audio output.

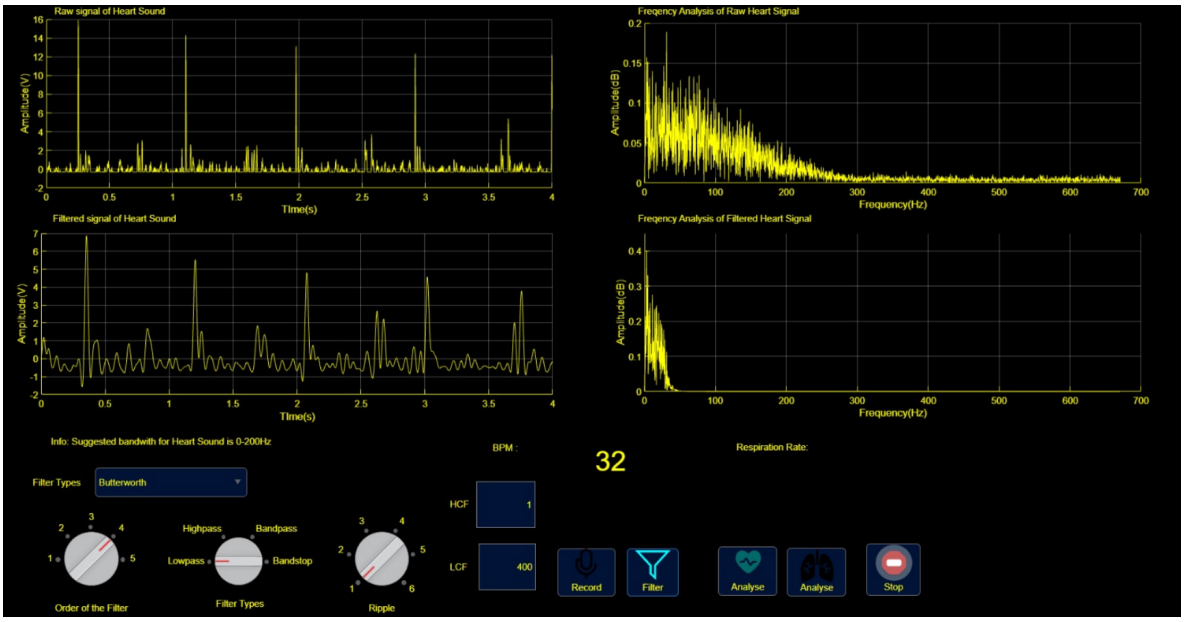

Figure-31 (PCB Heartbeat Plotting in GUI)

4.3 FUSION 360 BOX DESIGN

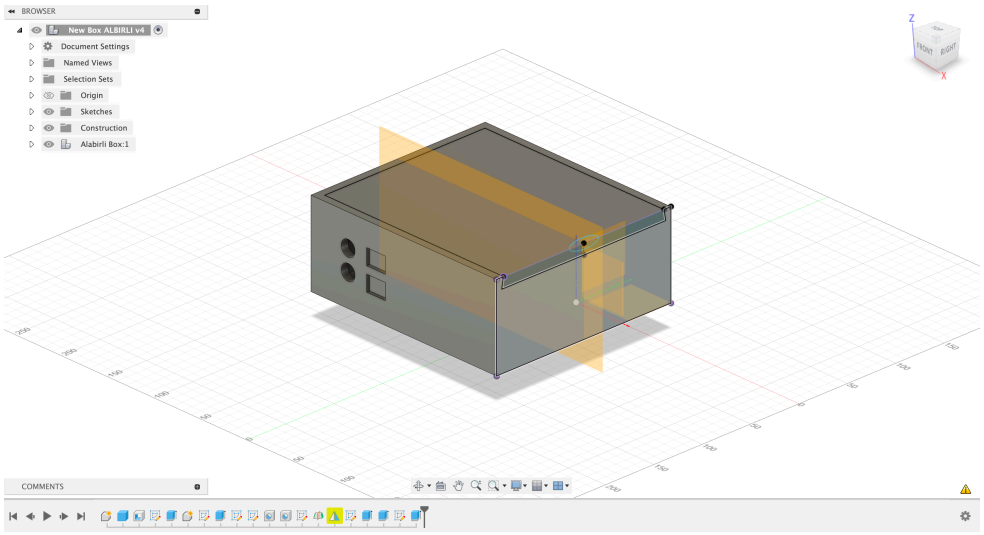

Figure-32 (Fusion 360 3D Box)

In the Fusion 360 design, a 3D Box with specific dimensions has been methodically crafted. The overall dimensions of the box are 170mm x 170mm, with height of 80mm (170x170x80). The box comprises two distinct parts: the lid and the box itself. The lid measures 159mm x 164mm and has a height of 10mm. Notably, the lid features an 85-degree incline, facilitating a smooth sliding mechanism onto the corresponding section of the box. This inclined design enhances the functionality of the lid, allowing for easy access to the contents of the box while providing a secure closure.

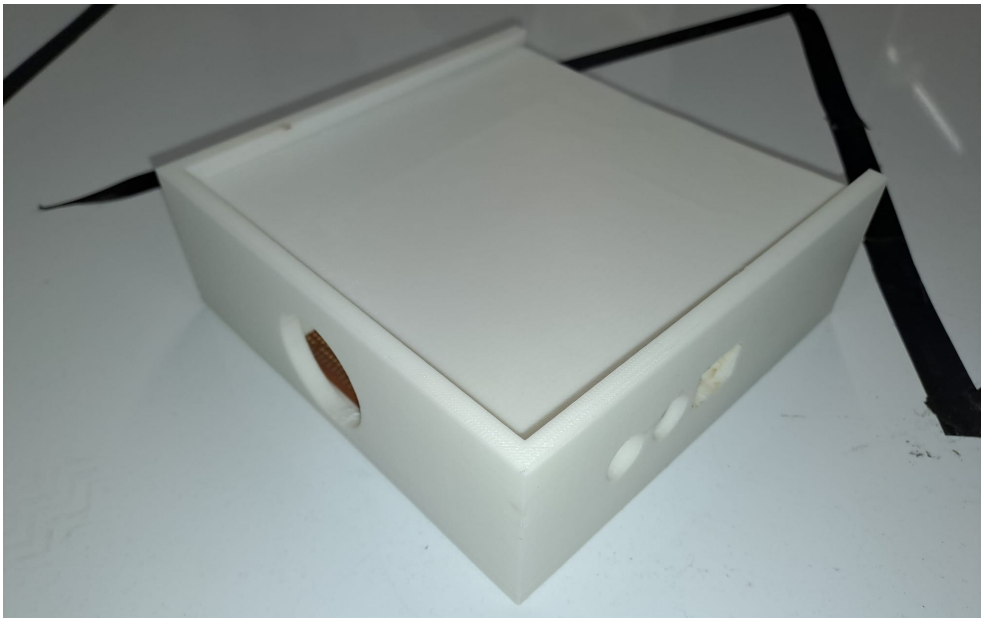

Figure-33 (Printed 3D Box)

A 20mm x 10mm rectangular cutout accommodates an audio jack, while two circular cutouts with a radius of 40mm each facilitate for wire connections for integrated circuitry. The entire 3D box design is intended to be fabricated using a 3D printing machine situated in the Printer Lab within the Abdullah Gül University Laboratory Building.

5. Conclusions

In conclusion, the successful completion of the electronic stethoscope project represents a conclusion of rigorous experimentation, careful design considerations, and innovative implementation. The core of the system involved the utilization of an electret microphone, complemented by an amplifier circuit and the integration of low-pass and high-pass filters. This intricate combination was meticulously calibrated, taking into account key parameters such as cut-off frequency, gain, and the relevant frequency range and as well as a buffer to separate filters. The result was a robust system that provided an accurate representation of heart and lung sounds while effectively minimizing interference from ambient noise.

Throughout the project, several experiments were conducted to refine the design and functionality of the electronic stethoscope. These experiments not only contributed to the overall success of the project but also served as invaluable learning experiences. In the spirit of transparency, it is noteworthy to mention instances where challenges were encountered and overcome. For example, oversights such as forgetting to connect a common ground were encountered, leading to temporary setbacks. These failures, however, served as important lessons, prompting a thorough review of the project's wiring and connections, ultimately contributing to the enhancement of the overall system reliability.

In the pursuit of flexibility and practicality, two distinct circuits were designed. The first circuit was implemented on both a breadboard and a stripboard, providing a tangible and flexible testing platform. The second circuit, featuring a more permanent and streamlined design, was meticulously crafted using PCB designing techniques with Altium Designer. This dual approach not only showcased adaptability in the prototyping phase but also emphasized the importance of scalability and reliability in the final product.

Furthermore, the project's success was significantly augmented by the implementation of a Graphical User Interface (GUI) in MATLAB®. Serving as the central control hub, the GUI played a pivotal role in real-time signal monitoring and analysis through the Signal's Fast Fourier Transform (FFT).

In essence, the electronic stethoscope project stands as a demonstration to the collaboration of theoretical knowledge, hands-on experimentation, and the perseverance to overcome

challenges. The incorporation of fails and subsequent corrective measures not only enriched the learning experience but also underscored the importance of a systematic and iterative approach to engineering projects.

Resources

1- https://www.electronics-tutorials.ws/filter/filter_8.html
2- https://www.mathworks.com/help/signal/ref/butter.html
3- Dliou, A., Latif, R., Laaboubi, M., Maoulainine, F., & Elouaham, S. (2012, November). Noised abnormal ECG signal analysis by combining EMD and Choi-Williams techniques. In 2012 IEEE International Conference on Complex Systems (ICCS) (pp. 1-5). IEEE.
4- Hossain, A., Uddin, S., Rahman, P., Anee, M. J., Rifat, M. M. H., & Uddin, M. M. (2022). Wavelet and Spectral Analysis of Normal and Abnormal Heart Sound for Diagnosing Cardiac Disorders. *BioMed Research International*, *2022*.